

noted

ON-PREMISES MLOPS COCKPIT



isi

UNIVERSITY
INSTITUTE
OF LISBON

DEVELOPER MANUAL

Table of Contents

1. Introduction	5
1.1 Who this manual is for	5
1.2 How to read this manual.....	5
1.3 Prerequisites	5
1.4 The 30-second tour	6
2. Anatomy of a noted project	7
2.1 Directory conventions	7
2.2 NOTED.md for mounted projects	8
2.3 Python environment	9
2.4 Git and .gitignore.....	9
3. Configuring with Hydra	11
3.1 What you need to know	11
3.2 Authoring configuration files	11
3.3 Reading cfg in a notebook.....	13
3.4 Reading cfg in a plain Python module	14
3.5 Reading cfg in an Airflow DAG task	15
3.6 The Configuration Composer	16
3.7 Per-run configuration archival.....	17
3.8 Override cheat sheet	17
4. Tracking data with DVC	19
4.1 What you need to know	19
4.2 Initializing DVC for a project.....	19
4.3 Adding a dataset	19
4.4 Deriving a subset dataset.....	20
4.5 Reading a DVC-tracked file from code	21
4.6 Logging the dataset hash alongside a run.....	22
4.7 Reproducing a past dataset version.....	22
5. Recording runs with MLflow	24
5.1 What you need to know	24
5.2 The Run Manager in a notebook.....	24
5.3 Running outside the Run Manager (plain Python)	26
5.4 Registering and promoting models	27
5.5 Searching runs by tag	29
6. Orchestrating with Airflow	30
6.1 What you need to know	30

6.2 The canonical DAG skeleton	30
6.3 Attaching the Hydra bundle from a DAG	33
6.4 Scheduling conventions	35
6.5 Triggering a DAG from code	35
7. Data quality & drift with Evidently	36
7.1 What you need to know	36
7.2 Writing a quality report	36
7.3 Writing a drift report	37
7.4 As an Airflow task	38
7.5 Reading snapshots back	39
8. Lineage: the three hashes	40
8.1 What you need to know	40
8.2 Logging the three hashes	40
8.3 The hydra/ artifact bundle	42
8.4 Finding related runs by lineage	42
9. Serving a registered model	44
9.1 What you need to know	44
9.2 Deploying from the noted UI	44
9.3 Calling /predict from an external client	44
9.4 Health and schema inspection	46
10. Writing custom skills	47
10.1 What you need to know	47
10.2 Triggers	48
10.3 Priority and token budget	48
10.4 References	48
10.5 Debugging a skill	48
11. Worked example: a notebook-based project	50
12. Worked example: a Python-plus-DAG project	54
12.1 src/paths.py (convention)	54
12.2 src/data/ingest.py	54
12.3 src/features/engineer.py	55
12.4 src/features/split.py	55
12.5 src/models/train.py	56
12.6 src/evaluation/promote.py	58
12.7 dags/train_pipeline.py	60
12.8 Directory recap	63
13. Reference	65

13.1 Environment variables	65
13.2 Endpoint cheat sheet.....	65
13.3 MLflow tag conventions	66
13.4 MLflow param conventions	66
13.5 Evidently tag conventions	66
13.6 Troubleshooting	66

1. Introduction

1.1 Who this manual is for

This manual is written for software engineers and ML practitioners building projects that run inside noted. It assumes a running noted stack and a working knowledge of Python, pandas, and a common ML framework (TensorFlow, PyTorch, scikit-learn, or XGBoost). It does **not** cover how noted itself is built.

Read this manual when:

- You are starting a new project on noted and want to know the conventions.
- You are integrating one of noted's subsystems (Hydra, MLflow, DVC, Airflow, Evidently) into an existing project and need the minimum viable wiring.
- You want to keep the same pipeline code working from a notebook, a plain Python module, and an Airflow DAG - noted was designed to make that single-source-of-truth feasible.

If you intend to modify noted itself, the companion platform-internals manual covers that.

1.2 How to read this manual

Each chapter covers one concern - configuration, data, tracking, orchestration, quality, lineage, serving - and follows a consistent shape:

1. **What you need to know** - the minimum mental model.
2. **In a notebook** - code that runs in a notebook cell inside noted.
3. **In a plain Python module** - code that runs outside noted, same effect.
4. **In an Airflow DAG** - code that runs unattended on a schedule.

Not every chapter has all three variants (orchestration is always a DAG), but where a behavior differs between execution contexts, both are shown side-by-side.

Copy-pasteable code is the point. All examples target a fictional project called `demand_forecast` - a SKU-level daily-demand model for a retail chain. The domain is kept generic so you can substitute your own.

1.3 Prerequisites

A running noted stack. The default docker-compose setup exposes:

Service	Port	Hostname (inside compose)
noted web UI	8123	noted
MLflow	5000	mlflow
MinIO (S3)	9000	minio
MinIO console	9001	minio

Service	Port	Hostname (inside compose)
Airflow UI	8080	airflow-apiserver
Evidently	8009 (host) / 8000 (container)	evidently
Knowledge Graph	5523	noted-graph
Model serving	5522	noted-serving

Ensure pandas, scikit-learn, mlflow, hydra-core, omegaconf, dvc[s3], evidently, and your ML framework of choice are installed in your project's env (see Chapter 2).

1.4 The 30-second tour

Every project you build in noted follows the same shape:

```

your_project/
  config/
    config.yaml          # defaults list + inlined leaves
  data/
  model/
data/                  # DVC-tracked datasets live here
src/                  # plain Python modules (domain logic)
  data/
  features/
  models/
  evaluation/
dags/                 # Airflow DAGs (optional)
notebooks/           # exploratory + training notebooks
NOTED.md             # project metadata (optional)
requirements.txt

```

You configure with Hydra, version data with DVC, track experiments with MLflow, orchestrate with Airflow, monitor with Evidently. noted wraps all five with UI conveniences (the Composer, the Run Manager, the Knowledge Graph, the Time Machine, the Registry), but the underlying artifacts - YAML files, .dvc pointers, MLflow runs, DAG code - are yours. Nothing in this manual is noted-proprietary; everything works if you ever have to take your project off noted.

2. Anatomy of a noted project

2.1 Directory conventions

A noted project is a directory on disk. It can live inside `data/projects/` (internal projects, managed by the running noted instance) or be mounted from an external path via `NOTED.md` (external projects, kept under their own git repo).

The minimum viable tree:

```
demand_forecast/  
  config/  
    config.yaml  
  data/  
    default.yaml  
  model/  
    default.yaml  
  src/  
    __init__.py  
  notebooks/  
    welcome.ipynb  
  requirements.txt
```

Expanded, a realistic project:

```
demand_forecast/  
  .dvc/config                                # MinIO remote configuration  
  .dvcignore  
  .gitignore  
  .noted/                                    # noted-private project state (agents, views)  
  config/  
    config.yaml                             # defaults + inline leaves  
  data/  
    full_history.yaml  
    last_year.yaml  
  model/  
    xgboost.yaml  
    linear.yaml  
  scaler/  
    standard.yaml  
    minmax.yaml  
  dags/  
    train_pipeline.py                       # Airflow DAG (optional)  
  data/  
    daily_sales_full.csv
```

```
daily_sales_full.csv.dvc      # committed to git
daily_sales_last_year.csv
daily_sales_last_year.csv.dvc
notebooks/
  01_explore.ipynb
  02_train.ipynb
src/
  __init__.py
data/
  __init__.py
  ingest.py
  filter.py
features/
  __init__.py
  engineer.py
models/
  __init__.py
  train.py
  predict.py
evaluation/
  __init__.py
  metrics.py
  promote.py
tests/
  __init__.py
  test_ingest.py
NOTED.md
requirements.txt
README.md
```

The **convention**, not a rule enforced by noted: config/ holds Hydra YAMLs, data/ holds DVC-tracked binaries, src/ holds your importable Python modules, dags/ holds Airflow DAG definitions, notebooks/ holds notebooks. noted scans these directories for its various UIs (the Data Catalog tab looks at data/*.dvc, the Configuration Composer reads config/, the Pipelines view watches dags/). If you deviate from the convention, the scanners will not find your content.

2.2 NOTED.md for mounted projects

If your project lives outside data/projects/ (typically in its own git repo, checked out anywhere on the host), mount it by adding an entry to data/NOTED.md:

```
---
mounts:
  - name: demand_forecast
```

```
host_path: /home/you/repos/demand_forecast
```

On next stack restart, a `data/docker-compose.mounts.yml` is regenerated from this frontmatter and merged into the compose stack. The mounted directory appears as a project in noted's left sidebar with the name you chose.

Mounted projects have two advantages: the git history stays where you expect it (no git-submodule awkwardness inside `data/projects/`), and you can use your normal editor / toolchain outside noted while still seeing the project inside noted's UI.

2.3 Python environment

Each project has a Python environment. noted manages them in `data/environments/python/3.12/{env_name}/`. For a new project, create an env through the UI (Environments tab, New Environment) or via uv:

```
uv venv data/environments/python/3.12/demand_forecast
uv pip install --python data/environments/python/3.12/demand_forecast/bin/python \
    -r requirements.txt
```

`requirements.txt` for the examples in this manual:

```
pandas
numpy
scikit-learn
xgboost
mlflow
hydra-core
omegaconf
dvc[s3]
evidently
matplotlib
```

Kernels started by noted for this project will use this env automatically once the env is associated with the project (set it in the Kernel dropdown in the notebook toolbar).

2.4 Git and `.gitignore`

Commit everything **except**:

- `data/*.csv`, `data/*.parquet` (or whatever binary formats live there) - these are DVC-tracked; commit the `.csv.dvc` pointer file instead.
- `.venv/`, `__pycache__`, `.pytest_cache/`, `.ipynb_checkpoints/`.

- `data/environments/` if you store the env inside the project (typical when the env path is project-relative).

A minimum `.gitignore`:

```
__pycache__/  
*.pyc  
.venv/  
.ipynb_checkpoints/  
.pytest_cache/  
  
# Data: tracked via DVC  
data/*.csv  
data/*.parquet  
data/*.feather  
!data/*.dvc
```

Commit `config/`, `data/`, `src/`, `notebooks/`, `*.dvc`, `NOTED.md`, and `requirements.txt`. That is enough for another developer to clone your project, `dvc pull`, run the env creation, and get a working replica.

3. Configuring with Hydra

3.1 What you need to know

Hydra lets you build a configuration by composing YAML fragments. A **defaults list** at the top of config.yaml points at **groups** (directories); each group has one or more options (files). At compose time Hydra merges the chosen option of each group into a single tree. **Overrides** can then edit individual leaves without touching YAML.

The resolved configuration is an OmegaConf.DictConfig - a dict with attribute access (cfg.data.file), type coercion, interpolation (\${data.split.train}), and strict access (accessing a missing key raises, does not silently return None).

Two primitives are load-bearing:

- **Group composition** - changing one line in config.yaml (e.g. data: last_year -> data: full_history) swaps an entire YAML file into the merge.
- **Hash of the resolved config** - sha256(resolved_yaml) is the identity of a configuration. Two runs with identical hashes saw identical configurations; two runs with different hashes cannot be compared without first looking at the diff.

noted exposes this via the **Configuration Composer** (left sidebar, Hydra icon): a visual editor over group selections and overrides. It does not edit your YAML files; it edits the notebook's hydra_selections metadata, which drives what gets composed before the kernel runs.

3.2 Authoring configuration files

A conventional layout for demand_forecast:

config/config.yaml:

```
defaults:
  - data: full_history
  - model: xgboost
  - scaler: standard
  - _self_      # merge this file LAST so explicit values below win over group de
faults

seed: 42

training:
  test_size: 0.2
  cv_folds: 5
  early_stopping_rounds: 20
  eval_metric: rmse
```

```
logging:
  experiment_name: demand_forecast
  run_name: ${model.name}_${data.window}
```

The `- _self_` entry tells Hydra where to merge the current file relative to the group defaults. Hydra 1.1 introduced a composition-order change and emits `UserWarning: Defaults list is missing _self_ whenever the primary config carries top-level values alongside a defaults list (as ours does: seed, training.*, logging.*)`. Placing `_self_` at the end is the Hydra 1.1+ default: values defined directly in `config.yaml` override anything carried in from the group files. Placing it first reverses that precedence (Hydra 1.0 behavior). Keep it at the end unless you specifically want the group files to have the final word.

config/data/full_history.yaml:

```
name: full_history
file: data/daily_sales_full.csv
date_col: date
sku_col: sku
target_col: units_sold
feature_cols:
  - price
  - promo_flag
  - day_of_week
  - month
window:
  years: 3
```

config/data/last_year.yaml:

```
name: last_year
file: data/daily_sales_last_year.csv
date_col: date
sku_col: sku
target_col: units_sold
feature_cols:
  - price
  - promo_flag
  - day_of_week
  - month
window:
  years: 1
```

config/model/xgboost.yaml:

```
name: xgboost
params:
  n_estimators: 500
  max_depth: 6
  learning_rate: 0.05
  subsample: 0.8
  colsample_bytree: 0.8
  objective: reg:squarederror
```

config/model/linear.yaml:

```
name: linear
params:
  fit_intercept: true
  positive: false
```

config/scaler/standard.yaml:

```
name: standard
params:
  with_mean: true
  with_std: true
```

A composition with data: full_history, model: xgboost, scaler: standard produces a single resolved tree with `cfg.data.file = "data/daily_sales_full.csv"`, `cfg.model.params.n_estimators = 500`, and so on.

3.3 Reading `cfg` in a notebook

Inside noted, `cfg` is already in the kernel namespace when any cell runs. You do not import Hydra; you do not call `compose()`. noted reads the notebook's Hydra metadata, composes the tree in the backend, and injects `cfg` plus `__noted_hydra_hash__` via a silent kernel execution before your cell runs.

A notebook cell simply reads it:

```
# In a notebook cell inside noted - cfg is already defined.
print(cfg.data.file)
print(cfg.model.params.n_estimators)
print(__noted_hydra_hash__)

import pandas as pd
from pathlib import Path
PROJECT_ROOT = Path().resolve()
df = pd.read_csv(PROJECT_ROOT / cfg.data.file, parse_dates=[cfg.data.date_col])
```

The `__noted_hydra_hash__` string starts with sha256: and uniquely identifies the composed config. Log it as an MLflow tag on any run and you have a back-pointer from every artifact to its configuration.

To edit the composition interactively, open the Configuration Composer (left sidebar, Hydra icon). Change data: full_history to data: last_year in the Data dropdown, click Apply. The notebook is re-prepared; the next cell execution sees the new `cfg.data.file`. The Composer writes the selection into the notebook metadata so it survives save/reload.

3.4 Reading `cfg` in a plain Python module

Outside noted - a command-line script, a unit test, an Airflow task - you compose manually. Two patterns:

Pattern A: `hydra.compose()` in a standalone script.

Save the snippet below as `show_config.py` at the **project root** - the same directory that contains `config/`. The only invariant the script depends on is that its own file sits next to the `config/` folder. Do NOT name the file `hydra.py`: that shadows the installed package and from `hydra` import `compose` fails with a circular-import error.

```
# show_config.py (at the project root, next to config/)

from pathlib import Path
from hydra import compose, initialize_config_dir
from omegaconf import OmegaConf

CONFIG_DIR = str(Path(__file__).parent / "config")

def load_config(overrides=None):
    """Compose the project's Hydra configuration."""
    with initialize_config_dir(config_dir=CONFIG_DIR, version_base=None):
        return compose(config_name="config", overrides=overrides or [])

if __name__ == "__main__":
    import sys
    # Anything after "--" is a list of Hydra overrides.
    # e.g. python show_config.py -- training.cv_folds=10 model=linear
    overrides = sys.argv[sys.argv.index("--") + 1:] if "--" in sys.argv else []
    print(OmegaConf.to_yaml(load_config(overrides)))
```

Run it from the project root using that project's venv:

```
cd /app/mounts/demand_forecast
/app/data/environments/python/3.12/venv_demand_forecast/bin/python show_config.py
```

No `__init__.py` required, no `-m` needed, no `sys.path` adjustments. If you want to place the script in a subdirectory (e.g. `scripts/show_config.py`), change `Path(__file__).parent` to `Path(__file__).parent.parent` so the calculation still lands at the project root.

Chapter 12 introduces a richer `src/-package` layout for real pipelines - that layout moves this same logic into `src/cli/train.py` and is invoked as `python -m src.cli.train` because the modules import each other. Start with the standalone form above; graduate to the package form only when you actually need cross-module imports.

Pattern B: `@hydra.main` for a pure-CLI entry point.

Save as `train_pure.py` at the project root (alongside `config/`):

```
# train_pure.py (at the project root)
import hydra
from omegaconf import DictConfig

@hydra.main(config_path="config", config_name="config", version_base=None)
def main(cfg: DictConfig) -> None:
    print(cfg.data.file)
    print(cfg.model.params.n_estimators)

if __name__ == "__main__":
    main()
```

Run with overrides as bare positional arguments (`@hydra.main` parses them natively, no `--` separator needed):

```
python train_pure.py data=last_year training.cv_folds=10
```

Pattern A is more flexible (you can call `load_config()` from notebooks, tests, DAGs). Pattern B is more idiomatic if the module really is just a CLI.

3.5 Reading `cfg` in an Airflow DAG task

Airflow tasks do not inherit noted's kernel injection. Compose manually inside the task using Pattern A:

```
# dags/train_pipeline.py (snippet)
from airflow.decorators import dag, task
```

```

from datetime import datetime
import os

@task
def compose_config(overrides: list[str] | None = None) -> dict:
    """Compose Hydra config inside a DAG task. Returns a plain dict
    because Airflow serializes XCom values as JSON."""
    from hydra import compose, initialize_config_dir
    from omegaconf import OmegaConf
    config_dir = os.path.join(
        os.environ.get("NOTED_PROJECTS_DIR", "/opt/noted/projects"),
        "demand_forecast",
        "config",
    )
    with initialize_config_dir(config_dir=config_dir, version_base=None):
        cfg = compose(config_name="config", overrides=overrides or [])
    return OmegaConf.to_container(cfg, resolve=True) # dict, not DictConfig

@dag(
    dag_id="demand_forecast_train",
    schedule="0 2 * * *",
    start_date=datetime(2024, 1, 1),
    catchup=False,
    tags=["demand_forecast", "training"],
)
def demand_forecast_train():
    cfg = compose_config(overrides=["data=full_history", "model=xgboost"])
    # downstream tasks reference cfg via XCom; see Chapter 6 for full DAG

```

In the DAG, `cfg` is a plain dict (not a `DictConfig`) because XCom serializes to JSON. Downstream tasks receive it as a dict argument.

3.6 The Configuration Composer

Inside noted, every Hydra-using notebook gets a Composer panel. It renders:

1. One **dropdown per group** in `config.yaml`'s defaults list (data / model / scaler in our example).
2. One **input per override leaf** - every primitive in `config.yaml` that is not a group reference. `training.cv_folds`, `seed`, `training.early_stopping_rounds`, etc.
3. A **mode toggle** between Local (compose from config/ in the working tree) and Experiment Run (compose from an archived bundle, see Section 3.7).
4. An **Apply** button that writes the current selection to the notebook metadata.

The Composer does **not** edit YAML files. It edits `notebook.metadata.noted.hydra_selections`, a nested dict:

```
{
  "group_selections": {"data": "last_year", "model": "xgboost"},
  "overrides": {"training.cv_folds": 10, "seed": 123}
}
```

When you run the notebook, noted reads this, composes the config, injects cfg, and runs your cells.

3.7 Per-run configuration archival

Every MLflow run created inside noted's Run Manager (or from an Airflow DAG that follows the convention) receives a hydra/ artifact subtree containing the full configuration the run saw:

```
hydra/
  config.yaml                # copy of the project's config.yaml at run time
  data/full_history.yaml
  data/last_year.yaml
  model/xgboost.yaml
  model/linear.yaml
  scaler/standard.yaml
  scaler/minmax.yaml
  selections.json            # {group_selections, overrides} that were applied
  resolved.yaml             # the composed output
```

plus two MLflow tags:

- `noted.hydra_config_hash = sha256:<hash>`
- `noted.project_id = demand_forecast`

With these you can, at any future point, pick a run in the Composer (Experiment Run mode), load its archived bundle, and re-execute against the *exact* configuration the run used. This is the Time Machine pattern and is the primary reason the archival exists.

For DAG-generated runs, you have to log the bundle yourself. Chapter 6 shows the pattern.

3.8 Override cheat sheet

In the Composer UI: type a new value into the input field.

In a notebook's URL or a CLI invocation: use Hydra override syntax. Examples below assume you have the `show_config.py` script from §3.4 sitting at the project root. Anything after the `--` separator is a Hydra override:

```
# Override a leaf
python show_config.py -- training.cv_folds=10

# Swap a group
python show_config.py -- data=last_year

# Multiple overrides
python show_config.py -- data=last_year model=linear seed=7

# Add a key that wasn't in the schema
python show_config.py -- +training.verbose=True

# Remove a key
python show_config.py -- ~logging.run_name
```

Overrides apply on top of the defaults list. The Composer persists only overrides that differ from the schema's defaults; unchanged leaves are not written to metadata.

If you instead use the Pattern B script from §3.5 (train_pure.py with @hydra.main), overrides go as bare positional arguments without the -- separator: python train_pure.py training.cv_folds=10. Use whichever form matches the script you have.

4. Tracking data with DVC

4.1 What you need to know

DVC (Data Version Control) versions large files using git-committed pointer files. The actual bytes live in a remote object store (MinIO, S3, anything S3-compatible). You commit `dataset.csv.dvc` - a small YAML file containing the dataset's md5, path, and size - and `dvc pull` reconstitutes the bytes from the remote.

Four things to know:

1. A `.dvc` file is a small YAML pointer. It is safe to commit; the data itself is `.gitignore`d.
2. The md5 in a `.dvc` file is an **identity**. Two datasets with the same md5 are bit-for-bit identical.
3. `dvc pull` checks the md5 against the remote and downloads missing bytes; it never re-uploads duplicates.
4. Remote configuration lives in `.dvc/config`. For a noted project, the remote points at MinIO inside the compose network.

4.2 Initializing DVC for a project

From the project root:

```
dvc init
dvc remote add -d minio s3://noted-dvc
dvc remote modify minio endpointurl http://minio:9000
dvc remote modify minio access_key_id admin
dvc remote modify minio secret_access_key password
```

Adjust credentials for your deployment. After this, `.dvc/config` reads:

```
[core]
  remote = minio
['remote "minio"']
  url = s3://noted-dvc
  endpointurl = http://minio:9000
  access_key_id = admin
  secret_access_key = password
```

Commit `.dvc/config` and `.dvcignore` to git. Do not commit `.dvc/tmp/` or `.dvc/cache/`.

4.3 Adding a dataset

Place the file under `data/`:

```
# From project root
dvc add data/daily_sales_full.csv
```

DVC creates `data/daily_sales_full.csv.dvc` and adds `data/daily_sales_full.csv` to `.gitignore`. The pointer file looks like:

```
outs:
- md5: 8d1f3c2e0a9b4d7f6e5a4b3c2d1e0f9a
  size: 43200000
  hash: md5
  path: daily_sales_full.csv
```

Push the bytes to the remote:

```
dvc push
```

Commit the pointer file to git:

```
git add data/daily_sales_full.csv.dvc .gitignore
git commit -m "Add full sales dataset (DVC)"
```

Another developer who clones the repo now runs `dvc pull` to materialize the actual CSV.

4.4 Deriving a subset dataset

When you want a filtered version (e.g. last year only), use a normal Python script and `dvc add` the output. `noted` does not use DVC pipelines; Airflow is the pipeline tool. A one-shot filter, saved at the project root next to `data/`:

```
# filter_last_year.py (at the project root)

from pathlib import Path
import pandas as pd

PROJECT_ROOT = Path(__file__).parent
df = pd.read_csv(PROJECT_ROOT / "data" / "daily_sales_full.csv", parse_dates=["date"])
df[df["date"].dt.year == 2024].to_csv(
    PROJECT_ROOT / "data" / "daily_sales_last_year.csv", index=False
)
```

Run it from the project root, then track the output:

```
python filter_last_year.py
dvc add data/daily_sales_last_year.csv
dvc push
git add data/daily_sales_last_year.csv.dvc
git commit -m "Add last-year subset dataset"
```

Both `data/full_history.yaml` and `data/last_year.yaml` in your Hydra config can now reference their respective CSV files; switching `data:` in the Composer swaps the dataset.

4.5 Reading a DVC-tracked file from code

In any execution context, the read is a normal file read. The caller is responsible for ensuring `dvc pull` has been run before the file is expected to exist.

In a notebook cell (noted sets the kernel CWD to the project root, so `Path()` resolves there):

```
import pandas as pd
from pathlib import Path

df = pd.read_csv(Path() / cfg.data.file, parse_dates=[cfg.data.date_col])
```

In a plain Python script at the project root (`load_data.py`):

```
# Load_data.py (at the project root)

from pathlib import Path
import pandas as pd
from hydra import compose, initialize_config_dir
from omegaconf import OmegaConf

PROJECT_ROOT = Path(__file__).parent

with initialize_config_dir(config_dir=str(PROJECT_ROOT / "config"), version_base=
None):
    cfg = compose(config_name="config")

data_path = PROJECT_ROOT / cfg.data.file
if not data_path.exists():
    raise FileNotFoundError(f"{data_path} not found. Run 'dvc pull' from the proj
ect root.")

df = pd.read_csv(data_path, parse_dates=[cfg.data.date_col])
df = df.sort_values(cfg.data.date_col).reset_index(drop=True)
print(df.head())
```

Run it as `python load_data.py` from the project root.

In an Airflow DAG task: see Chapter 6.

4.6 Logging the dataset hash alongside a run

To record which dataset a training run saw, log the md5 as an MLflow param and a tag. noted does this automatically for Run-Manager-launched runs in a notebook, but you must do it manually when launching from an Airflow task or a plain script.

The md5 lives inside the `.dvc` pointer file (a small YAML next to your dataset). Reading it is a one-line `yaml.safe_load`:

```
# Log_dvc_lineage.py (at the project root)

from pathlib import Path
import mlflow
import yaml

PROJECT_ROOT = Path(__file__).parent
DATA_FILE = "data/daily_sales_full.csv"  # relative to project root

dvc_pointer = PROJECT_ROOT / f"{DATA_FILE}.dvc"
md5 = yaml.safe_load(dvc_pointer.read_text())["outs"][0]["md5"]

mlflow.set_tracking_uri("http://mlflow:5000")
mlflow.set_experiment("demand_forecast")
with mlflow.start_run():
    mlflow.log_param("dvc_data_hash", md5)
    mlflow.set_tag("dvc.data_hash", md5)
    mlflow.set_tag("dvc.data_file", DATA_FILE)
    # ... rest of training ...
```

The resulting run carries `dvc.data_hash` as a tag. Any later search across runs can filter by this tag to find “every run that trained on this exact dataset”. Chapter 8 promotes this pattern into a reusable `record_lineage()` helper inside `src/lineage/`.

4.7 Reproducing a past dataset version

Every time you `dvc add` a changed file, a new md5 lands in the `.dvc` pointer and a new commit lands in git. To reproduce the exact bytes a past run saw:

```
# Find the commit that pinned the desired md5
git log -p data/daily_sales_full.csv.dvc

# Check out that version of the pointer
```

```
git checkout <commit_sha> -- data/daily_sales_full.csv.dvc  
  
# Pull the bytes matching that md5  
dvc pull data/daily_sales_full.csv.dvc
```

noted's Data Catalog tab surfaces the version history and provides a Checkout button that runs the same sequence.

5. Recording runs with MLflow

5.1 What you need to know

MLflow stores per-run metadata: parameters (immutable), metrics (time-series scalars), tags (mutable key/value), and artifacts (any file). Runs live inside named **experiments**. A separate **registry** catalogs model versions with movable **alias** pointers (@champion, @challenger, @staging).

In noted, runs are typically started by the **Run Manager** (which opens a run, installs hooks, and closes it after your cells finish). When you write code outside the Run Manager (a plain script, a DAG task), you open and close runs manually.

The important MLflow APIs:

```
import mlflow

mlflow.set_tracking_uri("http://mlflow:5000")  # inside compose
mlflow.set_experiment("demand_forecast")

with mlflow.start_run(run_name="xgboost_full_history") as run:
    mlflow.log_param("seed", 42)
    mlflow.log_metric("rmse", 12.3, step=1)
    mlflow.set_tag("owner", "team_x")
    mlflow.xgboost.log_model(model, artifact_path="model", signature=sig)
    print(run.info.run_id)
```

log_param is immutable (second call with the same key raises). log_metric accumulates (use step for time-series). set_tag is mutable (second call overwrites).

5.2 The Run Manager in a notebook

Inside noted, the Run Manager panel (right sidebar) wraps one or more notebook cells in an MLflow run. Clicking Run:

1. Opens a new MLflow run with the configured experiment/run names.
2. Installs a metric-streaming hook so mlflow.log_metric(...) calls also surface in noted's Live Metrics panel in real time.
3. Sets noted.hydra_config_hash, dvc.data_hash (if a DVC-tracked dataset is resolved from cfg.data.file), noted.project_id, and noted.git_commit tags.
4. Executes the selected cells.
5. Closes the run.

Your notebook cells do **not** call start_run or end_run when run via the Run Manager; noted already did it. They just call log_param, log_metric, log_model as if inside an active run.

A training cell inside the Run Manager:

```
# In a notebook cell inside noted, executed via the Run Manager.
# cfg is injected; an MLflow run is already open.

import mlflow
import xgboost as xgb
from sklearn.metrics import mean_squared_error
import numpy as np

X_train, y_train, X_val, y_val, X_test, y_test = split_data(df, cfg)

mlflow.log_params({
    "n_estimators": cfg.model.params.n_estimators,
    "max_depth": cfg.model.params.max_depth,
    "learning_rate": cfg.model.params.learning_rate,
    "cv_folds": cfg.training.cv_folds,
    "seed": cfg.seed,
})

model = xgb.XGBRegressor(
    **cfg.model.params,
    random_state=cfg.seed,
    early_stopping_rounds=cfg.training.early_stopping_rounds,
)
model.fit(
    X_train, y_train,
    eval_set=[(X_val, y_val)],
    verbose=False,
)

preds = model.predict(X_test)
rmse = float(np.sqrt(mean_squared_error(y_test, preds)))
mlflow.log_metric("test_rmse", rmse)

# Log target stats so inference clients can inverse-transform predictions.
mlflow.log_param("target_mean", float(y_train.mean()))
mlflow.log_param("target_std", float(y_train.std()))

# Log the model with a signature for serving.
from mlflow.models import infer_signature
sig = infer_signature(X_train, model.predict(X_train[:5]))
mlflow.xgboost.log_model(model, artifact_path="model", signature=sig)
```

Note that `log_metric` calls during `model.fit(...)`'s per-iteration evaluation would also stream live if you attached a callback that calls `mlflow.log_metric` per boosting round. XGBoost does not do this by default; a callback:

```
from xgboost.callback import TrainingCallback

class MlflowLogger(TrainingCallback):
    def after_iteration(self, model, epoch, evals_log):
        for data_name, metric_dict in evals_log.items():
            for metric_name, values in metric_dict.items():
                mlflow.log_metric(f"{data_name}_{metric_name}", values[-1], step=epoch)
        return False

model = xgb.XGBRegressor(**cfg.model.params, callbacks=[MlflowLogger()])
```

With the callback installed and the Run Manager active, noted's Live Metrics chart updates every iteration.

5.3 Running outside the Run Manager (plain Python)

In a standalone script, open and close the run yourself. This script uses synthetic data so it runs without any DVC or Hydra setup; once you understand the MLflow flow you can swap the synthetic block for a real `pd.read_csv` of your own data. Save it at the project root as `train_standalone.py`:

```
# train_standalone.py (at the project root)

import numpy as np
import mlflow
from sklearn.datasets import make_regression
from sklearn.linear_model import Ridge
from sklearn.metrics import mean_squared_error
from sklearn.model_selection import train_test_split
from mlflow.models import infer_signature

# 1. Synthetic regression problem (replace with your own data later)
X, y = make_regression(n_samples=2000, n_features=12, noise=0.4, random_state=42)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# 2. Tell MLflow where to log to and which experiment to land in
mlflow.set_tracking_uri("http://mlflow:5000") # use http://localhost:5000 from the host
mlflow.set_experiment("demand_forecast")
```

```
# 3. Open a run, Log everything you care about, close it
with mlflow.start_run(run_name="ridge_baseline") as run:
    mlflow.set_tag("noted.project_id", "demand_forecast")

    params = {"alpha": 1.0, "fit_intercept": True}
    mlflow.log_params(params)
    mlflow.log_param("target_mean", float(y_train.mean()))
    mlflow.log_param("target_std", float(y_train.std()))

    model = Ridge(**params).fit(X_train, y_train)

    test_rmse = float(np.sqrt(mean_squared_error(y_test, model.predict(X_test))))
    mlflow.log_metric("test_rmse", test_rmse)

    sig = infer_signature(X_train, model.predict(X_train[:5]))
    mlflow.sklearn.log_model(model, artifact_path="model", signature=sig)

    print(f"Trained run: {run.info.run_id}  test_rmse: {test_rmse:.3f}")
```

Run it from the project root:

```
python train_standalone.py
```

Key differences from the notebook (Run Manager) path:

- You set the tracking URI explicitly. Inside the compose network, use `http://mlflow:5000`; from the host, `http://localhost:5000`.
- You open and close the run with `mlflow.start_run(...)`.
- You set noted's lineage tags by hand. Inside the Run Manager these are automatic.
- You log `target_mean` / `target_std` yourself so the serving client can inverse-transform predictions (Chapter 9).

Chapter 12 expands this into a full pipeline that reads a real DVC-tracked dataset, applies feature engineering, and tags the run with the dataset's md5 + the git commit. The script above is the same MLflow flow, just with the integrations stripped out so you can see it end-to-end on its own.

5.4 Registering and promoting models

Once a run has a model artifact, register it and (optionally) promote it to @champion only if it beat the current champion. The script below is self-contained: pass it a `run_id` and it does everything in one shot. Save at the project root as `promote_run.py`:

```

# promote_run.py (at the project root)

import sys
import mlflow
from mlflow.tracking import MlflowClient

TRACKING_URI = "http://mlflow:5000" # or http://localhost:5000 from the host
MODEL_NAME = "demand_forecaster"
METRIC = "test_rmse"
LOWER_IS_BETTER = True

def main(run_id: str) -> None:
    mlflow.set_tracking_uri(TRACKING_URI)
    client = MlflowClient(tracking_uri=TRACKING_URI)

    # 1. Register the run's model artifact -> get a new version number
    new_version = int(
        mlflow.register_model(f"runs/{run_id}/model", MODEL_NAME).version
    )
    new_value = client.get_run(run_id).data.metrics.get(METRIC)
    if new_value is None:
        raise ValueError(f"Run {run_id} has no '{METRIC}' metric")

    # 2. Look up the current champion's metric (if any)
    try:
        champion = client.get_model_version_by_alias(MODEL_NAME, "champion")
        champion_value = client.get_run(champion.run_id).data.metrics.get(METRIC)
    except mlflow.exceptions.RestException:
        champion_value = None

    # 3. Decide and (maybe) move the alias
    is_better = (
        champion_value is None
        or (new_value < champion_value if LOWER_IS_BETTER else new_value > champion_value)
    )
    if is_better:
        client.set_registered_model_alias(MODEL_NAME, "champion", str(new_version))
        print(f"v{new_version} promoted to @champion ({METRIC}={new_value:.3f})")
    else:
        print(f"v{new_version} not promoted. {METRIC}={new_value:.3f}, "
              f"champion={champion_value:.3f}")

```

```
if __name__ == "__main__":
    if len(sys.argv) != 2:
        sys.exit("usage: python promote_run.py <run_id>")
    main(sys.argv[1])
```

Run it with the `run_id` printed by `train_standalone.py`:

```
python promote_run.py <run_id>
```

Aliases are the key idea. A serving client that reads `@champion` always sees the current best model. Rolling back to version 6 is one API call:

```
client.set_registered_model_alias("demand_forecaster", "champion", "6")
```

Chapter 12 packages this same logic into `src/evaluation/promote.py` with the registration and the better-than check split into separate functions, so both an Airflow task and an interactive script can call them.

5.5 Searching runs by tag

MLflow's search API works across runs by param, metric, or tag:

```
from mlflow.tracking import MlflowClient
client = MlflowClient(tracking_uri="http://mlflow:5000")

# ALL runs that saw a specific dataset md5:
runs = client.search_runs(
    experiment_ids=[client.get_experiment_by_name("demand_forecast").experiment_id],
    filter_string="tags.`dvc.data_hash` = '8d1f3c2e0a9b4d7f6e5a4b3c2d1e0f9a'",
    order_by=["metrics.test_rmse ASC"],
    max_results=20,
)
for run in runs:
    print(run.info.run_id, run.data.metrics.get("test_rmse"))
```

This is how noted's Compare panel, the Knowledge Graph, and the Composer's Experiment Run dropdown all filter runs to the relevant project/configuration/dataset scope.

6. Orchestrating with Airflow

6.1 What you need to know

Airflow runs DAGs - directed acyclic graphs of tasks - on schedules or on demand. A DAG file is a Python module that defines tasks (usually via the `@task` decorator) and their dependencies. Airflow's scheduler picks up DAGs from a watched directory (`/opt/airflow/dags` in noted's stack), parses them, and runs them.

Inside noted's compose stack:

- `airflow-apserver` serves the UI at `:8080`.
- `airflow-scheduler` schedules the DAGs.
- `airflow-worker` executes task instances (Celery executor).
- `airflow-triggerer` handles deferrable tasks.
- `postgres` + `redis` are the metadata DB and the Celery broker.

DAG files live in `<project>/dags/` and are mounted into the Airflow containers at startup.

6.2 The canonical DAG skeleton

A single-responsibility DAG: ingest, featurize, train, evaluate, promote. Each task is a thin wrapper over a module in `src/`.

```
# dags/train_pipeline.py
from airflow.decorators import task
from datetime import datetime
import os
import sys

# Adjust to wherever your project lives inside the Airflow worker container.
# This is the directory containing your config/, data/, and src/ folders.
PROJECT_ROOT = os.environ.get("NOTED_PROJECT_ROOT", "/opt/noted/projects/demand_forecast")

def _inject_project_path() -> None:
    """Make the project's src/ package importable from Airflow tasks."""
    if PROJECT_ROOT not in sys.path:
        sys.path.insert(0, PROJECT_ROOT)

@task
def compose_config(overrides: list[str] | None = None) -> dict:
    """Compose Hydra config; return as plain dict (XCom serialises as JSON)."""
```

```

from hydra import compose, initialize_config_dir
from omegaconf import OmegaConf
with initialize_config_dir(config_dir=f"{PROJECT_ROOT}/config", version_base=
None):
    cfg = compose(config_name="config", overrides=overrides or [])
    return OmegaConf.to_container(cfg, resolve=True)

```

@task

```

def ingest_data(cfg: dict) -> str:
    """Load + validate the dataset. Returns a path to a staged parquet."""
    _inject_project_path()
    from src.data.ingest import load_dataset
    from pathlib import Path

    df = load_dataset(
        Path(PROJECT_ROOT) / cfg["data"]["file"],
        date_col=cfg["data"]["date_col"],
    )
    staged = Path("/tmp") / f"ingested_{os.getpid()}.parquet"
    df.to_parquet(staged)
    return str(staged)

```

@task

```

def build_features(staged_path: str, cfg: dict) -> str:
    """Engineer features. Returns a path to a staged parquet."""
    _inject_project_path()
    from src.features.engineer import build_features as build
    import pandas as pd

    df = pd.read_parquet(staged_path)
    df = build(df, cfg)
    out = staged_path.replace("ingested_", "features_")
    df.to_parquet(out)
    return out

```

@task

```

def train_model_task(features_path: str, cfg: dict) -> str:
    """Train a model; return the MLflow run_id."""
    _inject_project_path()
    from src.models.train import train_from_features
    import pandas as pd

```

```

df = pd.read_parquet(features_path)
return train_from_features(df, cfg)

@task
def log_hydra_lineage(cfg: dict, run_id: str) -> None:
    """Attach the hydra/ artifact bundle and config hash tag to the run."""
    _inject_project_path()
    from src.lineage.hydra_bundle import attach_bundle_to_run
    attach_bundle_to_run(project_root=PROJECT_ROOT, cfg=cfg, run_id=run_id)

@task
def promote(run_id: str) -> dict:
    """Register the model and promote if it beats the current champion."""
    _inject_project_path()
    from src.evaluation.promote import register_model, promote_if_better
    new_version = register_model(run_id=run_id, model_name="demand_forecaster")
    return promote_if_better(run_id=run_id, new_version=new_version)

@dag(
    dag_id="demand_forecast_train",
    schedule="0 2 * * *",
    start_date=datetime(2024, 1, 1),
    catchup=False,
    tags=["demand_forecast", "training"],
    params={
        "data_config": "full_history",
        "model_config": "xgboost",
    },
    default_args={"owner": "ml-team", "retries": 0},
)
def train_pipeline():
    from airflow.operators.python import get_current_context
    ctx = get_current_context()
    overrides = [
        f"data={ctx['params']['data_config']}",
        f"model={ctx['params']['model_config']}",
    ]

    cfg = compose_config(overrides=overrides)
    ingested = ingest_data(cfg)

```

```

    featured = build_features(ingested, cfg)
    run_id = train_model_task(featured, cfg)

    # Fan out: Lineage + promotion run in parallel after training completes.
    log_hydra_lineage(cfg, run_id)
    promote(run_id)

dag_instance = train_pipeline()

```

This DAG imports its tasks from a `src/` package - the same package Chapter 12 builds end-to-end. Until you have that package in place, the DAG will fail to parse (the `src.*` imports won't resolve). Two paths:

- **For learning the Airflow + noted patterns:** read the DAG above for the orchestration shape, then jump to Chapter 12 to build the `src/` modules it depends on. Once Chapter 12 is in place, drop this DAG into `dags/` and trigger it.
- **For a minimum viable DAG you can run today:** replace each `_inject_project_path + from src.X import Y` with the equivalent code inlined in the task. Airflow will happily run a DAG whose tasks are all self-contained; you lose reuse but not correctness.

A trigger from the UI (or via `airflow dags trigger demand_forecast_train --conf '{"data_config":"last_year"}'`) runs the whole pipeline. The pipeline's MLflow run carries the same shape of tags as a Run-Manager-launched notebook run, so it appears in noted's Composer (Experiment Run dropdown), Registry, and Knowledge Graph.

6.3 Attaching the Hydra bundle from a DAG

The `log_hydra_lineage` task above calls `attach_bundle_to_run()`. The helper:

```

# src/Lineage/hydra_bundle.py
import hashlib
import json
import shutil
import tempfile
from pathlib import Path
import yaml
import mlflow
from mlflow.tracking import MlflowClient

def _sha256(text: str) -> str:
    return "sha256:" + hashlib.sha256(text.encode("utf-8")).hexdigest()

```

```

def _resolved_yaml(cfg: dict) -> str:
    """Stable YAML serialization of a resolved config dict."""
    return yaml.safe_dump(cfg, sort_keys=True, default_flow_style=False)

def attach_bundle_to_run(project_root: str, cfg: dict, run_id: str) -> None:
    """Upload the Hydra bundle as a hydra/ artifact subtree and tag the run
    with the config hash.

    The bundle contains:
    - config/                <- full config/ tree from the project
    - selections.json        <- the overrides + group selections
    - resolved.yaml         <- the composed output
    """

    pr = Path(project_root)
    config_src = pr / "config"
    if not config_src.is_dir():
        return

    with tempfile.TemporaryDirectory() as tmp:
        bundle = Path(tmp) / "hydra"
        bundle.mkdir()

        # Copy the entire config/ tree.
        shutil.copytree(config_src, bundle / "config", dirs_exist_ok=True)

        # Record selections (what groups + overrides produced this config).
        # For a DAG, selections live in the dag conf params; for simplicity
        # we store the whole resolved config under a synthetic 'overrides' key.
        selections = {
            "group_selections": {},
            "overrides": {},
            "origin": "airflow-dag",
        }
        (bundle / "selections.json").write_text(json.dumps(selections, indent=2))

        # Resolved YAML + hash.
        resolved_yaml = _resolved_yaml(cfg)
        (bundle / "resolved.yaml").write_text(resolved_yaml)
        config_hash = _sha256(resolved_yaml)

        # Upload to the run as hydra/.
        client = MlflowClient(tracking_uri="http://mlflow:5000")
        client.log_artifacts(run_id, str(bundle), artifact_path="hydra")

```

```
client.set_tag(run_id, "noted.hydra_config_hash", config_hash)
client.set_tag(run_id, "noted.project_id", pr.name)
```

After this task runs, the MLflow run has a hydra/ artifact subtree identical in shape to one produced by the Run Manager. It shows up in the Composer's Experiment Run dropdown, can be used as a Time Machine baseline, and feeds the Knowledge Graph's lineage edges.

6.4 Scheduling conventions

A few pragmatic defaults that work well inside noted:

- **Training DAGs:** schedule="0 2 * * *" (2 AM daily) or less often. Use catchup=False unless you deliberately want backfills.
- **Quality check DAGs:** hourly or every few hours. Usually a single-task DAG that runs an Evidently report.
- **Retry policy:** retries=0 for training tasks (reruns should be explicit), retries=2 for tasks that talk to external services (MLflow, MinIO) and can transiently fail.
- **Task-level timeouts:** set execution_timeout=timedelta(hours=2) on the train_model_task so a stuck kernel does not hold a worker forever.

6.5 Triggering a DAG from code

```
import requests

resp = requests.post(
    "http://airflow-apiserver:8080/api/v1/dags/demand_forecast_train/dagRuns",
    auth=("airflow", "airflow"),
    json={
        "conf": {
            "data_config": "last_year",
            "model_config": "linear",
        },
    },
)
resp.raise_for_status()
print(resp.json()["dag_run_id"])
```

From a notebook, use the same request. From inside another DAG task, prefer Airflow's TriggerDagRunOperator for proper dependency tracking.

7. Data quality & drift with Evidently

7.1 What you need to know

Evidently is a profiling and drift detection library. You hand it one or two DataFrames and it produces a **Report** containing **Metrics** - either wrapped in a **preset** (e.g. DataSummaryPreset, DataDriftPreset) or custom-assembled. The report can be saved as a **snapshot** in a **workspace** (persistent store) and viewed in Evidently's own UI.

Inside noted's stack, the Evidently service runs at `http://evidently:8000` and is reached via RemoteWorkspace. Workspace data is persisted in the `evidently-data` named volume, so snapshots survive stack restarts.

Three presets cover 90% of use cases:

- DataSummaryPreset - single-dataset stats (distributions, nulls, uniques). Run this on training data to establish a baseline.
- DataDriftPreset - compare two datasets. Run this on train vs. test (or train vs. prod) to detect distribution shift.
- RegressionPreset - model-output statistics. Run this on predictions to monitor model quality over time.

7.2 Writing a quality report

In a notebook cell or a plain Python module:

```
from evidently import Report, Dataset
from evidently.presets import DataSummaryPreset
from evidently.ui.workspace import RemoteWorkspace

def write_quality_snapshot(df, project_name: str = "demand_forecast",
                          tags: list[str] | None = None) -> str:
    """Profile a DataFrame and write a snapshot. Returns the snapshot id."""
    ws = RemoteWorkspace("http://evidently:8000")

    # Get or create the project.
    existing = [p for p in ws.list_projects() if p.name == project_name]
    project = existing[0] if existing else ws.create_project(project_name)

    report = Report(
        metrics=[DataSummaryPreset()],
        tags=tags or ["quality", project_name],
    )
    snapshot = report.run(current_data=Dataset.from_pandas(df))
    run_info = ws.add_run(project.id, snapshot, include_data=False)
```

```

    return run_info.id if hasattr(run_info, "id") else None

# Usage example: define df however you like (CSV read, query, sklearn dataset, ...)
import pandas as pd
df = pd.read_csv("data/daily_sales_full.csv", parse_dates=["date"])
write_quality_snapshot(df, tags=["quality", "demand_forecast", "daily"])

```

include_data=False stores aggregates only (small snapshots). Passing True embeds the raw rows and grows the volume quickly.

7.3 Writing a drift report

```

from evidently.presets import DataDriftPreset

def write_drift_snapshot(
    reference_df,
    current_df,
    project_name: str = "demand_forecast",
    run_id: str | None = None,
    tags: list[str] | None = None,
) -> str:
    ws = RemoteWorkspace("http://evidently:8000")
    existing = [p for p in ws.list_projects() if p.name == project_name]
    project = existing[0] if existing else ws.create_project(project_name)

    report = Report(
        metrics=[DataDriftPreset()],
        tags=tags or ["drift", project_name],
    )
    if run_id is not None:
        report.set_metadata({"run_id": run_id})

    snapshot = report.run(
        current_data=Dataset.from_pandas(current_df),
        reference_data=Dataset.from_pandas(reference_df),
    )
    run_info = ws.add_run(project.id, snapshot, include_data=False)
    return run_info.id if hasattr(run_info, "id") else None

```

The run_id metadata field is the **cross-link** to MLflow. When a drift snapshot is produced as part of a training run, stamping its MLflow run_id makes it possible to navigate from a drift finding back to the exact model that saw the drifted data. Always set it when you can.

7.4 As an Airflow task

These tasks assume the same `src/` package layout as the Chapter 6 DAG: `write_quality_snapshot` and `write_drift_snapshot` live in `src/quality/evidently_reports.py` (paste in the function bodies from §7.2 and §7.3). The `_inject_project_path()` helper is the one defined in the §6.2 DAG.

```
@task
def quality_report(features_path: str, cfg: dict) -> None:
    _inject_project_path()
    from src.quality.evidently_reports import write_quality_snapshot
    import pandas as pd

    df = pd.read_parquet(features_path)
    write_quality_snapshot(
        df,
        project_name=cfg["logging"]["experiment_name"],
        tags=["quality", cfg["logging"]["experiment_name"], "pipeline"],
    )

@task
def drift_report(features_path: str, cfg: dict, run_id: str) -> None:
    _inject_project_path()
    from src.quality.evidently_reports import write_drift_snapshot
    import pandas as pd

    df = pd.read_parquet(features_path)
    # Simple train/test split matching what the training task used.
    split = int(len(df) * (1 - cfg["training"]["test_size"]))
    write_drift_snapshot(
        reference_df=df.iloc[:split],
        current_df=df.iloc[split:],
        project_name=cfg["logging"]["experiment_name"],
        run_id=run_id,
        tags=["drift", cfg["logging"]["experiment_name"], "pipeline"],
    )
```

Wire into the §6.2 DAG:

```
def train_pipeline():
    # ... as before, up through:
    run_id = train_model_task(featured, cfg)

    # Quality + drift run in parallel after features are ready.
```

```
quality_report(featured, cfg)
drift_report(featured, cfg, run_id)

# Lineage + promotion continue on the main path.
log_hydra_lineage(cfg, run_id)
promote(run_id)
```

7.5 Reading snapshots back

To query snapshots programmatically (for an alerting loop, for example):

```
from evidently.ui.workspace import RemoteWorkspace

ws = RemoteWorkspace("http://evidently:8000")
project = next(p for p in ws.list_projects() if p.name == "demand_forecast")
snapshots = list(ws.list_snapshots(project.id))

for snap in snapshots[-5:]: # Latest 5
    print(snap.id, snap.metadata, snap.timestamp)
```

For inspection, open the Evidently tab in noted's left sidebar - it is the official Evidently UI proxied through nginx.

8. Lineage: the three hashes

8.1 What you need to know

Every trained model in noted is fully described by three hashes logged as tags on its MLflow run:

- **noted.hydra_config_hash** - sha256 of the resolved configuration YAML. Identifies *what the run was told to do*.
- **dvc.data_hash** (param dvc_data_hash) - md5 of the dataset the run trained on. Identifies *what bytes the run saw*.
- **noted.git_commit** (also mlflow.source.git.commit) - the git commit of the project at training time. Identifies *what code was running*.

With the three together, a reproduction attempt boils down to: check out the git commit, dvc pull the exact dataset version, compose the exact config, retrain. Any difference in outcome is then attributable to training-time nondeterminism (GPU scheduling, floating-point order) rather than to configuration or data drift.

8.2 Logging the three hashes

If you launch runs via the noted Run Manager, all three are logged automatically. If you launch outside noted (plain script or DAG), log them yourself.

```
# src/lineage/record_lineage.py
import subprocess
from pathlib import Path
import mlflow
import yaml
import hashlib

def _data_md5(dvc_pointer: Path) -> str:
    with open(dvc_pointer) as f:
        doc = yaml.safe_load(f)
    return doc["outs"][0]["md5"]

def _config_hash(resolved_yaml: str) -> str:
    return "sha256:" + hashlib.sha256(resolved_yaml.encode("utf-8")).hexdigest()

def _git_commit(repo_root: Path) -> str | None:
    try:
        return subprocess.check_output(
```

```

        ["git", "rev-parse", "HEAD"], cwd=repo_root
    ).decode().strip()
except subprocess.CalledProcessError:
    return None

def record_lineage(
    cfg,
    project_root: Path,
    data_path_rel: str,
    resolved_yaml: str,
    project_id: str,
) -> None:
    """Call this immediately after mlflow.start_run()."""
    # Config
    mlflow.set_tag("noted.hydra_config_hash", _config_hash(resolved_yaml))

    # Data
    pointer = project_root / f"{data_path_rel}.dvc"
    if pointer.exists():
        md5 = _data_md5(pointer)
        mlflow.log_param("dvc_data_hash", md5)
        mlflow.set_tag("dvc.data_hash", md5)
        mlflow.set_tag("dvc.data_file", data_path_rel)

    # Code
    sha = _git_commit(project_root)
    if sha:
        mlflow.set_tag("noted.git_commit", sha)
        mlflow.set_tag("mlflow.source.git.commit", sha)

    # Project scope
    mlflow.set_tag("noted.project_id", project_id)

```

Save the snippet above as `lineage.py` at the project root (or in `src/lineage/record_lineage.py` if you are following the Chapter 12 layout). Then call it at the top of every with `mlflow.start_run()`: block:

```

from pathlib import Path
from omegaconf import OmegaConf
from lineage import record_lineage # or: from src.Lineage.record_Lineage import record_Lineage

PROJECT_ROOT = Path(__file__).parent

```

```
with mlflow.start_run(run_name=cfg.logging.run_name) as run:
    record_lineage(
        cfg=cfg,
        project_root=PROJECT_ROOT,
        data_path_rel=cfg.data.file,
        resolved_yaml=OmegaConf.to_yaml(cfg),
        project_id="demand_forecast",
    )
# ... training ...
```

8.3 The hydra/ artifact bundle

In addition to the hash tag, noted stores the full *contents* of the config as an MLflow artifact subtree at `hydra/`. This is so a past run can be replayed even if the project's template files have changed since. The Run Manager does this automatically; Section 6.3 shows the DAG helper.

Bundle contents:

```
hydra/
  config/                # full copy of the project's config/ at training time
    config.yaml
    data/*.yaml
    model/*.yaml
    scaler/*.yaml
  selections.json        # {group_selections, overrides}
  resolved.yaml          # the composed output
```

To download a past run's bundle:

```
from mlflow.tracking import MlflowClient
from pathlib import Path

client = MlflowClient(tracking_uri="http://mlflow:5000")
local = Path("/tmp/old_hydra")
client.download_artifacts(run_id="abc123...", path="hydra", dst_path=str(local))
```

You can then re-compose from the downloaded `hydra/config/` using `initialize_config_dir()` with `config_dir=str(local / "hydra" / "config")`.

8.4 Finding related runs by lineage

Typical queries:

```
from mlflow.tracking import MlflowClient
client = MlflowClient(tracking_uri="http://mlflow:5000")

exp_id = client.get_experiment_by_name("demand_forecast").experiment_id

# All runs that trained on a specific dataset md5.
same_data = client.search_runs(
    [exp_id],
    filter_string="tags.`dvc.data_hash` = '8d1f3c2e0a9b4d7f6e5a4b3c2d1e0f9a'",
)

# All runs that saw a specific config hash.
same_config = client.search_runs(
    [exp_id],
    filter_string="tags.`noted.hydra_config_hash` = 'sha256:abc...'",
)

# All runs from a specific git commit.
same_code = client.search_runs(
    [exp_id],
    filter_string="tags.`noted.git_commit` = 'deadbeef...'",
)
```

With all three, a query like “find every run that used the same data + same config but a different git commit” is a single search with an AND. That is often the right question when diagnosing a regression: “*did this happen because the code changed?*”.

9. Serving a registered model

9.1 What you need to know

Once a model is registered in MLflow and the @champion alias points at it, noted's serving container can load and serve it. The serving contract is three HTTP endpoints:

- GET /health - returns the loader's status (idle, loading, ready, error) and metadata about the loaded model.
- GET /schema - returns the input/output schema of the loaded model (as inferred from the MLflow signature).
- POST /predict - runs inference on the request body and returns predictions.

The serving container resolves aliases on every load: when you deploy demand_forecaster@champion, it queries MLflow, finds the current version the alias points at, downloads the artifact, and loads it. Rolling back to an earlier version is client.set_registered_model_alias("demand_forecaster", "champion", "5") - no redeploy.

9.2 Deploying from the noted UI

In the Model Registry tab:

1. Select the model.
2. Select a version (usually the one tagged @champion).
3. Click Deploy. An NDJSON progress stream appears showing resolving -> downloading -> loading_model -> ready.
4. Click Try It. A form is rendered from the model's signature; fill it and click Predict.

The same flow works programmatically via the /api/serving/* endpoints exposed by the noted backend.

9.3 Calling /predict from an external client

A minimal client. Save as predict_client.py at the project root (or anywhere with httpx installed):

```
# predict_client.py
import httpx

class DemandForecastClient:
    def __init__(self, serving_url: str = "http://noted-serving:5522",
                 mlflow_url: str = "http://mlflow:5000",
                 model_name: str = "demand_forecaster"):
        self.serving_url = serving_url.rstrip("/")
```

```

self.mlflow_url = mlflow_url.rstrip("/")
self.model_name = model_name
self._scaler_mean: float | None = None
self._scaler_std: float | None = None

def _load_scaler_stats(self) -> None:
    """Read target_mean / target_std from the current champion's run."""
    if self._scaler_mean is not None:
        return
    alias = httpx.get(
        f"{self.mlflow_url}/api/2.0/mlflow/registered-models/alias",
        params={"name": self.model_name, "alias": "champion"},
    ).json()
    run_id = alias["model_version"]["run_id"]
    run = httpx.get(
        f"{self.mlflow_url}/api/2.0/mlflow/runs/get",
        params={"run_id": run_id},
    ).json()
    params = {p["key"]: p["value"] for p in run["run"]["data"]["params"]}
    self._scaler_mean = float(params["target_mean"])
    self._scaler_std = float(params["target_std"])

def predict(self, features: list[dict]) -> list[float]:
    """Send features, receive predictions in real (un-scaled) units."""
    self._load_scaler_stats()

    resp = httpx.post(
        f"{self.serving_url}/predict",
        json={"dataframe_records": features},
        timeout=30,
    )
    resp.raise_for_status()
    raw = resp.json()["predictions"]

    # If the model predicts in scaled space, inverse-transform.
    return [float(v) * self._scaler_std + self._scaler_mean for v in raw]

# Usage
client = DemandForecastClient()
predictions = client.predict([
    {"sku": "A-001", "price": 9.99, "promo_flag": 1, "day_of_week": 2, "month":
4},
    {"sku": "A-001", "price": 9.99, "promo_flag": 0, "day_of_week": 3, "month":

```

```
4},  
])  
print(predictions) # real-unit predictions, e.g. [142.5, 98.2]
```

The inverse-transform pattern (multiply by `target_std`, add `target_mean`) is the reason Chapter 5 recommended logging these two values as params on every training run. Without them, the client cannot convert scaled predictions back to human-readable units.

9.4 Health and schema inspection

```
import httpx  
  
serving_url = "http://noted-serving:5522" # use http://localhost:5522 from the  
host  
  
health = httpx.get(f"{serving_url}/health").json()  
print(health["status"])      # "ready" | "loading" | "idle" | "error"  
print(health["model_name"])  # "demand_forecaster"  
print(health["version"])     # "7"  
print(health["framework"])   # "xgboost"  
  
schema = httpx.get(f"{serving_url}/schema").json()  
print(schema["inputs"])      # [{"name": "price", "type": "double"}, ...]  
print(schema["outputs"])     # [{"name": "prediction", "type": "double"}]
```

A service that depends on the serving container should poll `/health` before sending traffic. A production client might refresh its scaler stats on alias change by polling the registered-model version and comparing to the cached `run_id`.

10. Writing custom skills

10.1 What you need to know

The noted AI assistant has a **skill registry** - a collection of small Markdown files under `data/skills/` that get auto-injected into the LLM's system prompt when their triggers match the current context. Writing a skill is how you teach the assistant about your project's conventions without changing code.

A skill is a directory containing a SKILL.md:

```
data/skills/demand-forecast-conventions/
  SKILL.md
  references/
    example_dag.py          # optional supporting files referenced by the skill
```

SKILL.md has YAML frontmatter and Markdown body:

```
---
name: demand-forecast-conventions
description: >-
  Conventions for the demand_forecast project: config structure, feature
  names, target scaler.
triggers: [project_demand_forecast]
priority: 2
max_tokens: 400
---
```

demand_forecast conventions

This project uses Hydra configuration with three groups: ``data``, ``model``, ``scaler``. All training runs log ``target_mean`` and ``target_std`` as params so inference clients can inverse-transform predictions.

- SKU column: ``sku``
- Date column: ``date``
- Target column: ``units_sold``
- Time-ordered split: train / val / test chronologically, never random.

When the user asks about adding a new model, remind them to:

1. Add a ``config/model/<name>.yaml`` with the model's hyperparameters.
2. Update ``src/models/train.py``'s dispatch to import the new model class.
3. Re-run the training pipeline via the Run Manager or the Airflow DAG.

10.2 Triggers

A skill's triggers list names conditions that cause auto-injection. The built-in vocabulary includes:

- `notebook_cell_selected` - a notebook cell is focused.
- `notebook_open` - any notebook is open.
- `project_<id>` - the active project matches the id.
- `mlflow_run_in_context` - a run is referenced in the conversation.
- `hydra_config_in_context` - a Hydra config is referenced.
- `dvc_file_selected` - a DVC-tracked file is in focus.
- `airflow_dag_selected` - an Airflow DAG is open.
- `file_extension_py` / `file_extension_ipynb` / `file_extension_yaml` - the active file's extension.

A skill with triggers: `[project_demand_forecast]` injects whenever the user is working in the `demand_forecast` project. A skill with triggers: `[project_demand_forecast, mlflow_run_in_context]` injects only when both are true (logical AND). Use multiple triggers to narrow scope.

10.3 Priority and token budget

`priority`: 1 (highest) through 5 (lowest) decides which skills win when the context has a token budget. Inject only what the LLM needs to do the current task well.

`max_tokens`: 400 is a soft cap; the registry stops injecting a skill mid-paragraph if it would overflow. Keep skills under ~500 tokens - they are meant to be surgical, not encyclopedic.

10.4 References

Files under the skill's `references/` subfolder can be opened by the assistant via a tool call (`get_skill_reference(skill_name, ref_path)`). A skill's body can hint at them:

```
For a full example of a training DAG, see references/example_dag.py in
this skill.
```

The LLM will fetch it on demand when the user asks “show me an example”.

10.5 Debugging a skill

If a skill is not being injected, open the AI assistant's debug panel (the bug icon on the Chat tab). Enable it and send a message. Each event logged includes the skills that matched and the triggers that fired. Common causes of non-injection:

- Trigger name typo in the frontmatter.

- The active context does not actually match (e.g. the file open is not what you expected).
- Another higher-priority skill has already consumed the token budget.
- The SkillRegistry has not reloaded since you added the file - restart the backend to pick it up.

11. Worked example: a notebook-based project

This chapter is a complete notebook for `demand_forecast`. It uses Hydra, DVC, MLflow, and Evidently - all the integrations in one ~10-cell notebook. Each cell is numbered and described. Use it as a template.

The notebook imports from a `src/` package (`src.data.ingest`, `src.features.engineer`, etc.). Those modules are fully defined in Chapter 12; build that package first if you want to actually run this notebook end-to-end. The earlier chapters (3-9) showed the same concepts as standalone scripts so you could learn each integration in isolation; this chapter and the next show how those concepts compose into a real production project.

Cell 1 - imports and project root.

```
import os
import sys
from pathlib import Path
import numpy as np
import pandas as pd

PROJECT_ROOT = Path().resolve()
sys.path.insert(0, str(PROJECT_ROOT))
print("Project:", PROJECT_ROOT.name)
```

Cell 2 - confirm cfg is injected.

```
# cfg is already in the kernel namespace, injected by noted before this cell.
print(f"Data file: {cfg.data.file}")
print(f"Model: {cfg.model.name}")
print(f"Config hash: {__noted_hydra_hash__}")
```

Cell 3 - seed.

```
import random
SEED = int(cfg.seed)
random.seed(SEED)
np.random.seed(SEED)
```

Cell 4 - load dataset (DVC-tracked).

```
from src.data.ingest import load_dataset

df = load_dataset(
    file_path=PROJECT_ROOT / cfg.data.file,
    date_col=cfg.data.date_col,
```

```
)  
print(f"{len(df):,} rows, {df[cfg.data.date_col].min()} to {df[cfg.data.date_col].max()}")  
df.head()
```

Cell 5 - feature engineering.

```
from src.features.engineer import build_features  
  
features_df = build_features(df, cfg)  
features_df.head()
```

Cell 6 - time-ordered split.

```
from src.features.split import split_data  
  
X_train, y_train, X_val, y_val, X_test, y_test = split_data(features_df, cfg)  
print(f"train: {len(X_train):,} | val: {len(X_val):,} | test: {len(X_test):,}")
```

Cell 7 - Evidently data-quality snapshot (pre-training).

```
from src.quality.evidently_reports import write_quality_snapshot  
  
write_quality_snapshot(  
    features_df,  
    project_name=cfg.logging.experiment_name,  
    tags=["quality", cfg.logging.experiment_name, "pre-training"],  
)
```

Cell 8 - train the model.

```
# Inside the Run Manager, an MLflow run is already open.  
# Log params, fit, log metrics + model.  
import mlflow  
import xgboost as xgb  
from sklearn.metrics import mean_squared_error  
from mlflow.models import infer_signature  
  
class MlflowIterationLogger(xgb.callback.TrainingCallback):  
    def after_iteration(self, model, epoch, evals_log):  
        for ds, metrics in evals_log.items():  
            for name, values in metrics.items():  
                mlflow.log_metric(f"{ds}_{name}", values[-1], step=epoch)
```

```

        return False

mlflow.log_params({
    **dict(cfg.model.params),
    "seed": cfg.seed,
    "cv_folds": cfg.training.cv_folds,
})
mlflow.log_param("target_mean", float(y_train.mean()))
mlflow.log_param("target_std", float(y_train.std()))

model = xgb.XGBRegressor(
    **cfg.model.params,
    random_state=cfg.seed,
    early_stopping_rounds=cfg.training.early_stopping_rounds,
    callbacks=[MlflowIterationLogger()],
)
model.fit(X_train, y_train, eval_set=[(X_val, y_val)], verbose=False)

val_rmse = float(np.sqrt(mean_squared_error(y_val, model.predict(X_val))))
test_rmse = float(np.sqrt(mean_squared_error(y_test, model.predict(X_test))))
mlflow.log_metric("val_rmse", val_rmse)
mlflow.log_metric("test_rmse", test_rmse)

sig = infer_signature(X_train, model.predict(X_train[:5]))
mlflow.xgboost.log_model(model, artifact_path="model", signature=sig)

print(f"val_rmse={val_rmse:.3f}  test_rmse={test_rmse:.3f}")

```

Cell 9 - Evidently drift snapshot (post-training).

```

from src.quality.evidently_reports import write_drift_snapshot

active_run = mlflow.active_run()
write_drift_snapshot(
    reference_df=features_df.iloc[:len(X_train) + len(X_val)],
    current_df=features_df.iloc[len(X_train) + len(X_val):],
    project_name=cfg.logging.experiment_name,
    run_id=active_run.info.run_id if active_run else None,
    tags=["drift", cfg.logging.experiment_name, "post-training"],
)

```

Cell 10 - register + promote.

```
from src.evaluation.promote import register_model, promote_if_better

active_run = mlflow.active_run()
if active_run is not None:
    new_version = register_model(run_id=active_run.info.run_id, model_name="demand_forecaster")
    result = promote_if_better(
        run_id=active_run.info.run_id,
        new_version=new_version,
    )
    print(result)
```

That is the whole notebook. Ten cells, each doing one thing. Open the Configuration Composer, change data: full_history to data: last_year, click Apply, run the Run Manager, and you have a parallel run trained on a different dataset - with all tags, all metrics, all artifacts, all lineage links filed in the right places.

12. Worked example: a Python-plus-DAG project

For production, you want the same pipeline in .py modules that Airflow can orchestrate. The code below mirrors the notebook from Chapter 11.

This chapter is also where the `src/` package referenced throughout earlier chapters is defined end-to-end. Each module here corresponds to a concept introduced earlier as a standalone script: §12.2 `ingest.py` mirrors §4.5; §12.5 `train.py` mirrors §5.3; §12.6 `promote.py` mirrors §5.4; the §12.7 DAG composes them all the way Chapter 6 hinted at. Read the standalone scripts first if you want to understand each piece in isolation; read this chapter to see how they fit together as a real project.

12.1 `src/paths.py` (convention)

Every script and DAG task in this chapter needs the project root to resolve config files, data files, and .dvc pointers. Centralise that in one place so no snippet has to count parent directories by hand.

```
# src/paths.py

from pathlib import Path

# This file lives at <project>/src/paths.py, so the project root is two parents u
p.
PROJECT_ROOT = Path(__file__).resolve().parent.parent
```

From any module under `src/` (or from a DAG that imports `src.*`), write `from src.paths import PROJECT_ROOT` and use the constant. If you later reorganise the tree, you change one file instead of every script.

12.2 `src/data/ingest.py`

```
from pathlib import Path
import pandas as pd

def load_dataset(file_path: Path, date_col: str = "date") -> pd.DataFrame:
    """Load a DVC-tracked CSV. Raises if the file is missing (indicating
a missing `dvc pull`)."
    if not file_path.exists():
        raise FileNotFoundError(
            f"{file_path} not found. Run 'dvc pull' from the project root."
        )
    df = pd.read_csv(file_path, parse_dates=[date_col])
    return df.sort_values(date_col).reset_index(drop=True)
```

12.3 `src/features/engineer.py`

Both `engineer.py` and `split.py` accept `cfg` either as a `DictConfig` (from Hydra inside noted) or a plain dict (from an Airflow DAG, because XCom serialises values as JSON). `OmegaConf.create(cfg)` normalises either input to `DictConfig` in one line, so the rest of the function can use attribute access uniformly.

```
import pandas as pd
from omegaconf import OmegaConf

def build_features(df: pd.DataFrame, cfg) -> pd.DataFrame:
    """Assemble the feature set that the model will consume."""
    cfg = OmegaConf.create(cfg) # accept dict or DictConfig

    out = df.copy()
    out["day_of_week"] = out[cfg.data.date_col].dt.dayofweek
    out["month"] = out[cfg.data.date_col].dt.month
    out["is_weekend"] = out["day_of_week"].isin([5, 6]).astype(int)

    # Lag features per SKU.
    out = out.sort_values([cfg.data.sku_col, cfg.data.date_col])
    out["lag_1"] = out.groupby(cfg.data.sku_col)[cfg.data.target_col].shift(1)
    out["lag_7"] = out.groupby(cfg.data.sku_col)[cfg.data.target_col].shift(7)
    out["roll_7_mean"] = (
        out.groupby(cfg.data.sku_col)[cfg.data.target_col]
        .shift(1)
        .rolling(window=7, min_periods=1)
        .mean()
        .reset_index(level=0, drop=True)
    )
    return out.dropna().reset_index(drop=True)
```

12.4 `src/features/split.py`

```
import pandas as pd
from omegaconf import OmegaConf

def split_data(df: pd.DataFrame, cfg):
    """Time-ordered train/val/test split.

    Returns (X_train, y_train, X_val, y_val, X_test, y_test).
    """
```

```

cfg = OmegaConf.create(cfg)
test_size = float(cfg.training.test_size)
val_size = test_size # reuse for simplicity

df_sorted = df.sort_values(cfg.data.date_col)
n = len(df_sorted)
n_test = int(n * test_size)
n_val = int(n * val_size)
n_train = n - n_val - n_test

feature_cols = list(cfg.data.feature_cols) + [
    "day_of_week", "month", "is_weekend", "lag_1", "lag_7", "roll_7_mean",
]
target_col = cfg.data.target_col

# Encode SKU as a category code - a crude embedding.
# A real system would use a proper encoder + persist it to the model artifact.
sku_codes = df_sorted[cfg.data.sku_col].astype("category").cat.codes
df_enc = df_sorted.assign(sku_code=sku_codes)
feature_cols = feature_cols + ["sku_code"]

def xy(start, end):
    sub = df_enc.iloc[start:end]
    X = sub[feature_cols].reset_index(drop=True)
    y = sub[target_col].reset_index(drop=True)
    return X, y

X_train, y_train = xy(0, n_train)
X_val, y_val = xy(n_train, n_train + n_val)
X_test, y_test = xy(n_train + n_val, n)
return X_train, y_train, X_val, y_val, X_test, y_test

```

12.5 [src/models/train.py](#)

```

import numpy as np
import pandas as pd
import mlflow
import xgboost as xgb
from sklearn.metrics import mean_squared_error
from mlflow.models import infer_signature
from omegaconf import OmegaConf

from src.data.ingest import load_dataset

```

```

from src.features.engineer import build_features
from src.features.split import split_data
from src.lineage.record_lineage import record_lineage
from src.paths import PROJECT_ROOT

def train_from_features(df: pd.DataFrame, cfg) -> str:
    """Core training loop. Returns the MLflow run_id.

    Caller is responsible for having `dvc pull`ed the data. cfg may be a
    DictConfig (from Hydra) or a plain dict (from a DAG); normalise on entry.
    """

    cfg = OmegaConf.create(cfg)
    X_train, y_train, X_val, y_val, X_test, y_test = split_data(df, cfg)

    mlflow.set_tracking_uri("http://mlflow:5000")
    mlflow.set_experiment(cfg.logging.experiment_name)

    with mlflow.start_run(run_name=cfg.logging.run_name) as run:
        record_lineage(
            cfg=cfg,
            project_root=PROJECT_ROOT,
            data_path_rel=cfg.data.file,
            resolved_yaml=OmegaConf.to_yaml(cfg),
            project_id="demand_forecast",
        )

        mlflow.log_params({
            **dict(cfg.model.params),
            "seed": int(cfg.seed),
            "cv_folds": cfg.training.cv_folds,
        })
        mlflow.log_param("target_mean", float(y_train.mean()))
        mlflow.log_param("target_std", float(y_train.std()))

        model = xgb.XGBRegressor(
            **cfg.model.params,
            random_state=int(cfg.seed),
            early_stopping_rounds=cfg.training.early_stopping_rounds,
        )
        model.fit(X_train, y_train, eval_set=[(X_val, y_val)], verbose=False)

        val_rmse = float(np.sqrt(mean_squared_error(y_val, model.predict(X_val))))

```

```

        test_rmse = float(np.sqrt(mean_squared_error(y_test, model.predict(X_test))))
        mlflow.log_metric("val_rmse", val_rmse)
        mlflow.log_metric("test_rmse", test_rmse)

        sig = infer_signature(X_train, model.predict(X_train[:5]))
        mlflow.xgboost.log_model(model, artifact_path="model", signature=sig)

    return run.info.run_id

def train(cfg) -> str:
    """Convenience wrapper: Load + feature-engineer + train."""
    cfg = OmegaConf.create(cfg)
    df = load_dataset(PROJECT_ROOT / cfg.data.file, date_col=cfg.data.date_col)
    df = build_features(df, cfg)
    return train_from_features(df, cfg)

if __name__ == "__main__":
    from src.cli.train import load_config
    cfg = load_config()
    run_id = train(cfg)
    print(f"Trained run: {run_id}")

```

12.6 [src/evaluation/promote.py](#)

Identical to Section 5.4. Included here for completeness.

```

import mlflow
from mlflow.tracking import MlflowClient

def register_model(run_id: str, model_name: str = "demand_forecaster") -> int:
    model_uri = f"runs/{run_id}/model"
    result = mlflow.register_model(model_uri, model_name)
    return int(result.version)

def get_champion_metric(model_name: str, metric: str) -> float | None:
    client = MlflowClient(tracking_uri="http://mlflow:5000")
    try:
        champion = client.get_model_version_by_alias(model_name, "champion")
    except Exception:

```

```

        return None

    run = client.get_run(champion.run_id)
    return run.data.metrics.get(metric)

def promote_if_better(
    run_id: str,
    new_version: int,
    metric: str = "test_rmse",
    model_name: str = "demand_forecaster",
    lower_is_better: bool = True,
) -> dict:
    client = MlflowClient(tracking_uri="http://mlflow:5000")
    new_run = client.get_run(run_id)
    new_value = new_run.data.metrics.get(metric)
    if new_value is None:
        raise ValueError(f"Run {run_id} has no '{metric}' metric.")
    champion_value = get_champion_metric(model_name, metric)

    promoted = False
    improvement = None
    if champion_value is None:
        promoted = True
    else:
        better = (new_value < champion_value) if lower_is_better else (new_value
> champion_value)
        if better:
            promoted = True
            improvement = (champion_value - new_value) / champion_value * 100
            if not lower_is_better:
                improvement = -improvement

    if promoted:
        client.set_registered_model_alias(model_name, "champion", str(new_versio
n))

    return {
        "promoted": promoted,
        "old_champion_value": champion_value,
        "new_value": new_value,
        "improvement_pct": improvement,
    }

```

12.7 dags/train_pipeline.py

```

from airflow.decorators import dag, task
from datetime import datetime, timedelta
import os
import sys

PROJECT_ROOT = os.environ.get("NOTED_PROJECT_ROOT", "/opt/noted/projects/demand_f
orecast")

def _inject_project_path():
    if PROJECT_ROOT not in sys.path:
        sys.path.insert(0, PROJECT_ROOT)

@task
def compose_config(overrides: list[str] | None = None) -> dict:
    from hydra import compose, initialize_config_dir
    from omegaconf import OmegaConf
    with initialize_config_dir(config_dir=f"{PROJECT_ROOT}/config", version_base=
None):
        cfg = compose(config_name="config", overrides=overrides or [])
        return OmegaConf.to_container(cfg, resolve=True)

@task
def ingest_data(cfg: dict) -> str:
    _inject_project_path()
    from src.data.ingest import load_dataset
    from pathlib import Path

    df = load_dataset(Path(PROJECT_ROOT) / cfg["data"]["file"],
                        date_col=cfg["data"]["date_col"])
    out = f"/tmp/ingested_{os.getpid()}.parquet"
    df.to_parquet(out)
    return out

@task
def build_features(staged_path: str, cfg: dict) -> str:
    _inject_project_path()
    from src.features.engineer import build_features as build

```

```
import pandas as pd
```

```
df = pd.read_parquet(staged_path)
df = build(df, cfg)
out = staged_path.replace("ingested_", "features_")
df.to_parquet(out)
return out
```

```
@task
```

```
def train_model_task(features_path: str, cfg: dict) -> str:
    _inject_project_path()
    from src.models.train import train_from_features
    import pandas as pd

    df = pd.read_parquet(features_path)
    return train_from_features(df, cfg)
```

```
@task
```

```
def log_hydra_lineage(cfg: dict, run_id: str) -> None:
    _inject_project_path()
    from src.lineage.hydra_bundle import attach_bundle_to_run
    attach_bundle_to_run(project_root=PROJECT_ROOT, cfg=cfg, run_id=run_id)
```

```
@task
```

```
def quality_report(features_path: str, cfg: dict) -> None:
    _inject_project_path()
    from src.quality.evidently_reports import write_quality_snapshot
    import pandas as pd
    df = pd.read_parquet(features_path)
    write_quality_snapshot(
        df,
        project_name=cfg["logging"]["experiment_name"],
        tags=["quality", cfg["logging"]["experiment_name"], "pipeline"],
    )
```

```
@task
```

```
def drift_report(features_path: str, cfg: dict, run_id: str) -> None:
    _inject_project_path()
    from src.quality.evidently_reports import write_drift_snapshot
    import pandas as pd
```

```

df = pd.read_parquet(features_path)
n = len(df)
split = int(n * (1 - cfg["training"]["test_size"] * 2))
write_drift_snapshot(
    reference_df=df.iloc[:split],
    current_df=df.iloc[split:],
    project_name=cfg["logging"]["experiment_name"],
    run_id=run_id,
    tags=["drift", cfg["logging"]["experiment_name"], "pipeline"],
)

```

@task

```

def promote_task(run_id: str) -> dict:
    _inject_project_path()
    from src.evaluation.promote import register_model, promote_if_better
    new_version = register_model(run_id=run_id, model_name="demand_forecaster")
    return promote_if_better(run_id=run_id, new_version=new_version)

```

@dag(

```

    dag_id="demand_forecast_train",
    schedule="0 2 * * *",
    start_date=datetime(2024, 1, 1),
    catchup=False,
    tags=["demand_forecast", "training"],
    params={
        "data_config": "full_history",
        "model_config": "xgboost",
    },
    default_args={
        "owner": "ml-team",
        "retries": 0,
        "execution_timeout": timedelta(hours=2),
    },
)

```

def train_pipeline():

```

    from airflow.operators.python import get_current_context
    ctx = get_current_context()
    overrides = [
        f"data={ctx['params']['data_config']}",
        f"model={ctx['params']['model_config']}",
    ]

```

```
cfg = compose_config(overrides=overrides)
ingested = ingest_data(cfg)
featured = build_features(ingested, cfg)
run_id = train_model_task(featured, cfg)

quality_report(featured, cfg)
drift_report(featured, cfg, run_id)
log_hydra_lineage(cfg, run_id)
promote_task(run_id)
```

```
dag_instance = train_pipeline()
```

12.8 Directory recap

After writing all the above, the project tree is:

```
demand_forecast/
  config/
    config.yaml
  data/
    full_history.yaml
    last_year.yaml
  model/
    xgboost.yaml
    linear.yaml
  scaler/
    standard.yaml
  data/
    daily_sales_full.csv.dvc
    daily_sales_last_year.csv.dvc
  dags/
    train_pipeline.py
  notebooks/
    01_explore.ipynb
    02_train.ipynb
  src/
    __init__.py
    cli/
      train.py
    data/
      __init__.py
      ingest.py
```

```
filter.py
dvc_lookup.py
features/
  __init__.py
  engineer.py
  split.py
models/
  __init__.py
  train.py
evaluation/
  __init__.py
  promote.py
lineage/
  __init__.py
  record_lineage.py
  hydra_bundle.py
quality/
  __init__.py
  evidently_reports.py
clients/
  __init__.py
  predict_client.py
tests/
  __init__.py
  test_ingest.py
  test_engineer.py
NOTED.md
requirements.txt
.dvc/config
.dvcignore
.gitignore
```

Running locally: `python -m src.models.train` (trains a model using the default config).

Running via Airflow: trigger `demand_forecast_train` in the Airflow UI with the desired `data_config` / `model_config` params.

Running in noted: open `notebooks/02_train.ipynb`, pick `config` in the Composer, click `Run` in the Run Manager.

All three paths produce MLflow runs with the same tags, the same lineage, and the same registered-model promotions. That is the payoff.

13. Reference

13.1 Environment variables

Set via `services/.env` (compose), by the frontend config, or via your own deployment automation.

Variable	Default	Purpose
NOTED_TERMINAL_SECRET	(unset)	Gate for terminal and Claude-model LLM access. If unset, terminal auth succeeds silently (dev mode).
ANTHROPIC_API_KEY	(unset)	Enables Claude backends in the AI assistant.
MINIO_ROOT_USER	admin	MinIO admin user; referenced by <code>.dvc/config</code> .
MINIO_ROOT_PASSWORD	password	MinIO admin password.
_AIRFLOW_WWW_USER_USERNAME	airflow	Airflow admin username.
_AIRFLOW_WWW_USER_PASSWORD	airflow	Airflow admin password.
AIRFLOW_UID	50000	UID for the Airflow container's owner; match your host user if you need write access to mounted dirs.
_PIP_ADDITIONAL_REQUIREMENTS	(empty)	Extra pip installs for Airflow workers. Use for project-specific packages the workers need to import.

13.2 Endpoint cheat sheet

Service	Inside compose	From host
noted backend	<code>http://noted:8123</code>	<code>http://localhost:8123</code>
MLflow	<code>http://mlflow:5000</code>	<code>http://localhost:5000</code>
MinIO S3	<code>http://minio:9000</code>	<code>http://localhost:9000</code>
MinIO console	<code>http://minio:9001</code>	<code>http://localhost:9001</code>
Airflow API	<code>http://airflow-apiserver:8080</code>	<code>http://localhost:8080</code>
Evidently	<code>http://evidently:8000</code>	<code>http://localhost:8009</code>
Knowledge Graph	<code>http://noted-graph:5523</code>	<code>http://localhost:5523</code>
Model serving	<code>http://noted-serving:5522</code>	<code>http://localhost:5522</code>

noted's backend exposes proxies under `/api/*` so the frontend does not need to know these URLs directly; the serving, graph, MLflow, and Airflow proxies are convenient for external clients that want a single origin.

13.3 MLflow tag conventions

Tag	Set by	Purpose
noted.hydra_config_hash	Run Manager / record_lineage()	Config identity (sha256:...).
noted.project_id	Run Manager / record_lineage()	Project filter in Composer / Knowledge Graph.
noted.git_commit	Run Manager / record_lineage()	Project git commit at training time.
mlflow.source.git.commit	Run Manager / record_lineage()	Standard MLflow git tag; mirror of above.
dvc.data_hash	Run Manager / record_lineage()	md5 of the DVC-tracked dataset.
dvc.data_file	Run Manager / record_lineage()	Project-relative path of the dataset.
instrumentation	Run Manager	"experiments" for Run Manager runs, "pipeline" for DAG runs (by convention).

13.4 MLflow param conventions

Param	Purpose
target_mean	Train-split mean of the regression target. Used by inference clients to inverse-transform scaled predictions.
target_std	Train-split std of the regression target. Same purpose.
dvc_data_hash	Duplicate of the tag, for faster filtering and easier display in the MLflow UI.

13.5 Evidently tag conventions

Tag	Purpose
quality	DataSummaryPreset / profiling reports.
drift	DataDriftPreset reports.
<project_name>	Scope to a single project.
<origin>	"pipeline" for DAG-generated, "notebook" for notebook-generated. Optional but useful for filtering.

13.6 Troubleshooting

cfg is not defined inside the notebook. You are probably running a cell before noted has injected it. Save the notebook, reopen it, and try again. If the problem persists, the notebook likely does not have Hydra metadata yet - open the Configuration Composer and click Apply once to initialize it.

Live metrics do not stream in the panel. The notebook is being executed via Run All (Play button) rather than the Run Manager. Run All bypasses the metrics monkey-patch. Use the Run Manager's Run button for tracked training.

Model Registry shows no @champion. Either no model has been promoted yet, or the registered model name in the UI does not match what `register_model()` used. Check the model name in `src/evaluation/promote.py`.

dvc pull fails with Unable to connect. MinIO is not reachable from wherever you are running the command. Inside the compose network, the endpoint is `http://minio:9000`. From your host, use `http://localhost:9000`. `.dvc/config` ships with the compose-internal hostname; when running DVC from your host, you may need a `.dvc/config.local` with the localhost URL.

Airflow task errors with ModuleNotFoundError: src. The worker container does not have your project on its `PYTHONPATH`. The DAG file must `sys.path.insert(0, PROJECT_ROOT)` before importing from `src`. The `_inject_project_path()` helper in Section 12.7 is the minimal fix.

Evidently snapshots vanish after a rebuild. Ensure the evidently-data named volume is declared in `docker-compose.yml` and mounted at `/app/workspace`. A bind mount is an alternative but a named volume is the supported path.

mlflow.register_model fails with RESOURCE_ALREADY_EXISTS. The registered model name already exists. This is fine; the call returns a new version under the existing name. The error only surfaces if you try to *create* via a different API. Confirm you are using `mlflow.register_model` (the convenience function) and not `MlflowClient.create_registered_model` unconditionally.

The Composer shows no override inputs. Only leaves in `config.yaml` itself are exposed as overrides at the moment; leaves inside group files are not. If you need a leaf to be overridable, inline it into `config.yaml` directly rather than nesting it inside a group file.

Serving /predict returns scaled numbers. The client is not applying the inverse transform. Read `target_mean` / `target_std` from the champion run's params (see Chapter 9.3) and multiply/add on the client side.